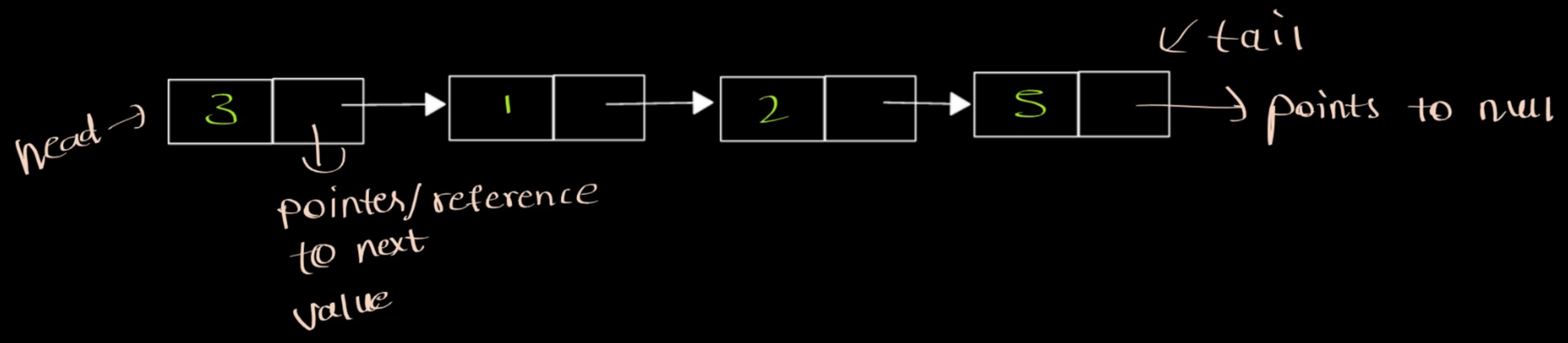
Linked List



⇒ It is a linear data structure contains nodes and each node will point to the next node.

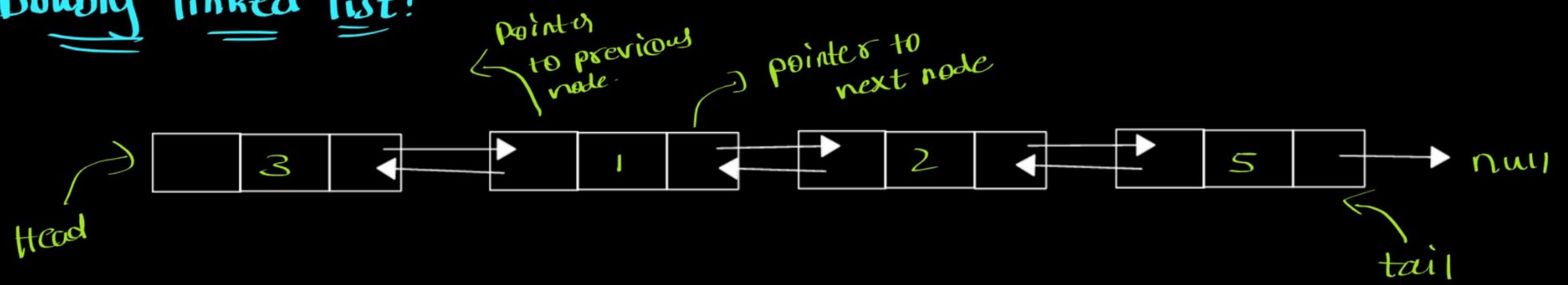
⇒ The starting point (or) generally LL is represented with HEAD using it we can access the whole LL.

⇒ The last node is TAIL and it points to null

⇒ Each node contains data and pointer/reference to next node.

This is singly linked list we have another type called Doubly linked list.

Doubly linked list:



⇒ Similar to linked list but here we have access to previous node as well

⇒ Single node of doubly linked list contains previous and next pointers and data/value.

* Difference between Arrays and Linked list:

Arrays

⇒ Have contingous memory with index.

3	1	2	5

⇒ fixed size

⇒ Accessing/fetching an element is complex O(n)

(Here we need traverse each and element one by one to get that element)

⇒ Insertion/Deletion is better than array and easy O(n)

(Here we need to create new node and change pointers)

Linked list

⇒ not contingous but point towards the next node using reference.

⇒ Dynamic in nature.

⇒ Accessing/fetching is easy. O(1)

(using index we can easily access)

⇒ Insertion/Deletion can be tricky. O(n)

(Here we need to shift elements and make space for new element)

⇒ extra memory

(Here we need to use 2 variables ↓ (singly LL) and 3 variables (doubly LL) to store one value/Data)

⇒ memory efficient

(Here memory is fixed and we get contingous memory for 1 value we use only one reference)

General usecase (not a playbook):

(1) Access elements by index fast → Array

(2) Insertion/deletion especially at head and tail → Linked list

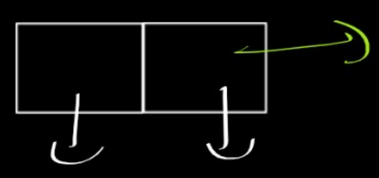
(3) Frequent traverse/manipulation → Linkedlist

(4) Unknown size (or) avoid resizing overhead → linkedlist

(5) Static-site and memory efficient → array

Design Linked List

⇒ Creating a node:



if nothing is there points to null

Value

pointer to next node

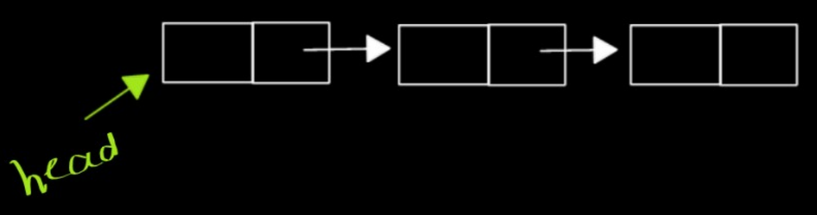
```
function Node(val)
{
    this.val = val;
    this.next = null;
}
```

```
let newNode = new Node(5);
```
↓ Keyword

⇒ creating a linked list: We generally represents linked list with head

if Linked list is empty it points to null and size is 0.

```
function MyLinkedList( )
{
    this.head = null;
    this.size = 0;
}
```



Note: this is just initialization

Just like we do for variables like let a=0;

⇒ need to create some more function to make it proper linked list

MyLinkedList() Initializes the MyLinkedList object.

int get(int index) Get the value of the indexth node in the linked list. If the index is invalid, return -1.

void addAtHead(int val) Add a node of value val before the first element of the linked list. After the insertion, the new node will be the first node of the linked list.

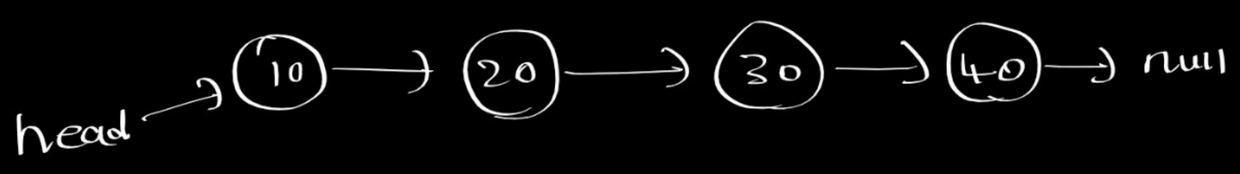
void addAtTail(int val) Append a node of value val as the last element of the linked list.

void addAtIndex(int index, int val) Add a node of value val before the indexth node in the linked list. If index equals the length of the linked list, the node will be appended to the end of the linked list. If index is greater than the length, the node will not be inserted.

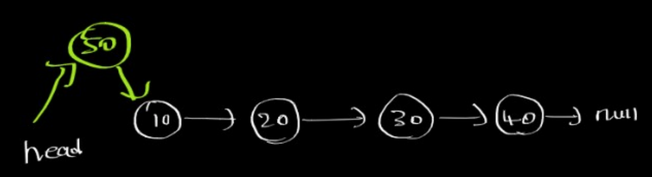
void deleteAtIndex(int index) Delete the indexth node in the linked list, if the index is valid.

Adding at head, tail and index

addAtHead(val):



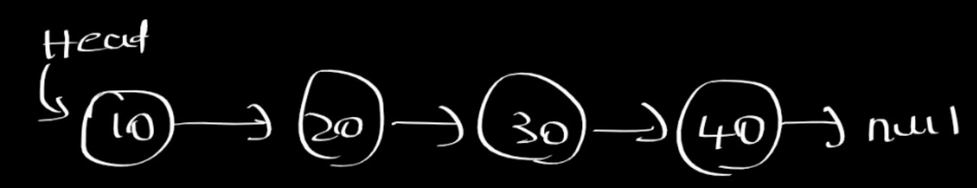
Algo: ① Create new node
② Point it to what head is pointing to
③ Now make head points to new node.
④ increase size.



code:

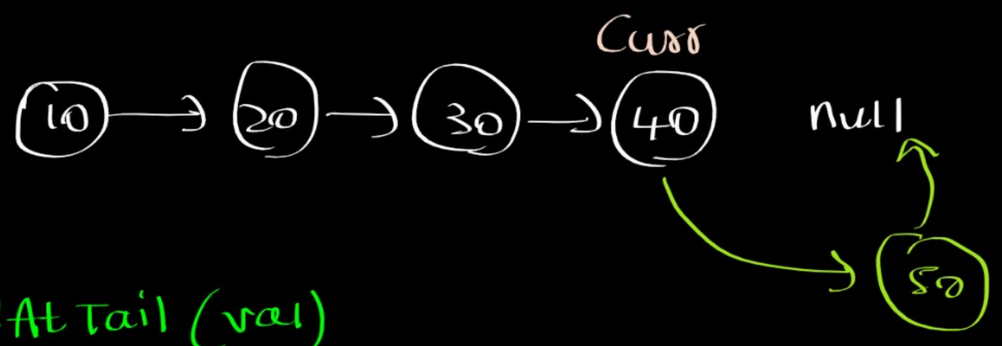
```
function addAtHead(val)
{
    let newNode = new Node(val);
    newNode.next = this.head;
    this.head = newNode;
    this.size++
}
```

addAtTail(val):



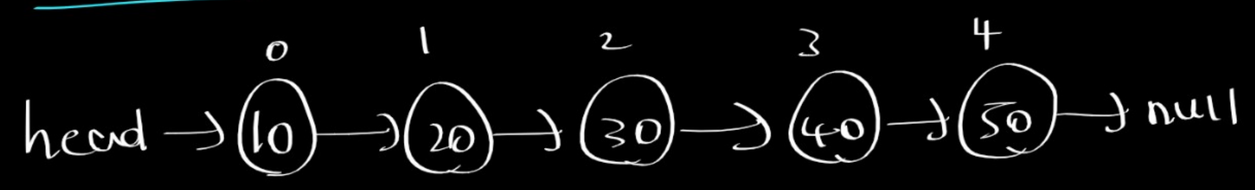
Algo: ① Create a new node that we want to add.
② first we need to reach the end to add at end.
③ from the current node point towards new node
④ As the new node points to null we don't need to explicitly do it.
⑤ Corner case: what if Linked list is empty (this.head = null)
Just point head to newly created node.

⑥ increment the size after these operations.



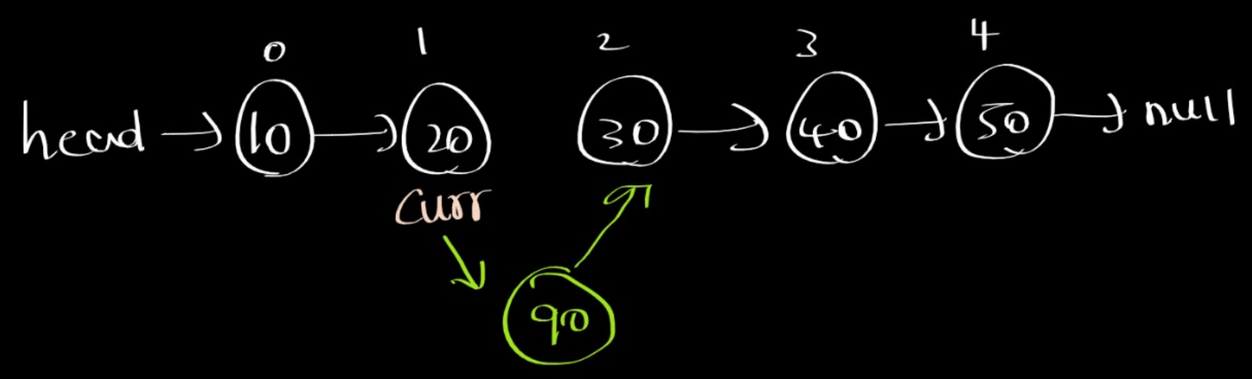
Code:
```
function addAtTail(val)
{
    let newNode = new Node(val);
    if(this.head ==null)
    {
        this.head = newNode;
    }
    else
    {
        let curr = this.head;
        while(curr.next != null)
        {
            curr = curr.next;
        }
        curr.next = newNode;
    }
    this.size++
}
```

8. addAtIndex(index, val):



Algo: ① Create a new node that we want to add
② Create a curr variable which points to starting node.
③ move before the index
④ point newNode.next to current node next
⑤ point current node next to new node
⑥ corner cases: ① what if the size is 0 and index is also 0
② what if the size and index is equal.
simply we can add at head
simply we can add at tail
⑦ finally increment the size.

Ex: addAtIndex(2, 90);

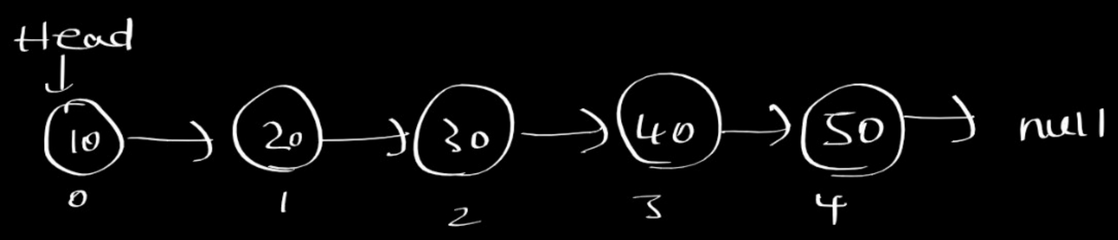


Code:

```
function addAtIndex(index, val)
{
    let newNode = new Node(val);
    if(index<0 || index > this.size) return;
    if( index == 0 )
    {
        addAtHead(val);
        return;
    }
else if(index == this.size)
{
    addAtTail(val);
     return;
 }
 else
 {
    let curr = this.head;
    for(let i=0; i< index-1; i++)
    {
        curr = curr.next;
    }
    newNode.next = curr.next;
    curr.next = newNode;
 }
this.size++;
}
```

Getting a value and deletion

ex. get(index):

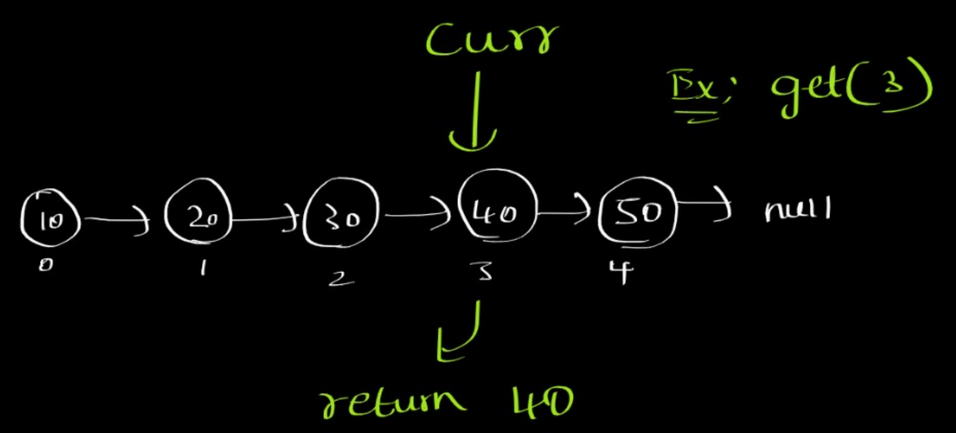


Algo: (1) reach to specific index
(2) return the data/value
(3) Corner cases: ⇒ index out of range → return -1

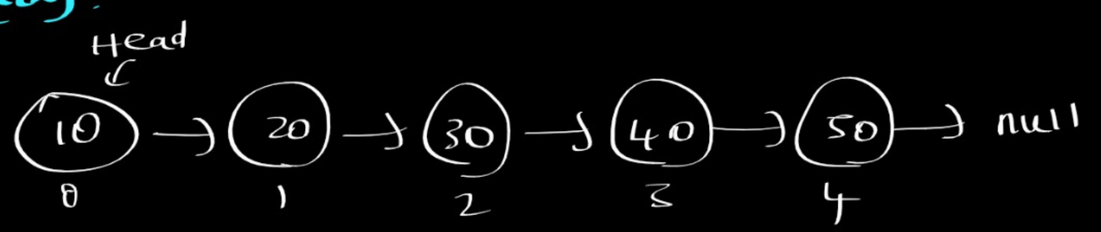
Code:

```
function get(index)
{
    if(index<0 || index > this.size-1) return -1;
```

```
let curr = this.next
for(let i=0 ; i< index ; i++)
{
    curr = curr.next;
}
return curr.val
}
```



K- deleteAtIndex(index):



Algo: (1) we can't delete some by reaching that node we need to reach just before the desired index.

(2) when we reach just simply point to next to next node (2 steps ahead) that means simply removing the pointer to next node and pointing to next to next node.

(3) Corner cases: (1) deleting head → just move the head to next node (so that we can remove first node)

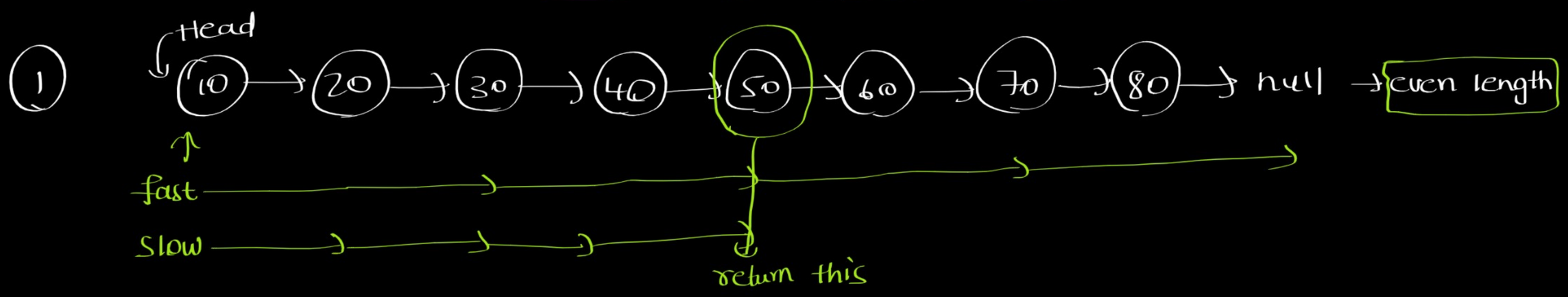
at Tail → our logic works fine

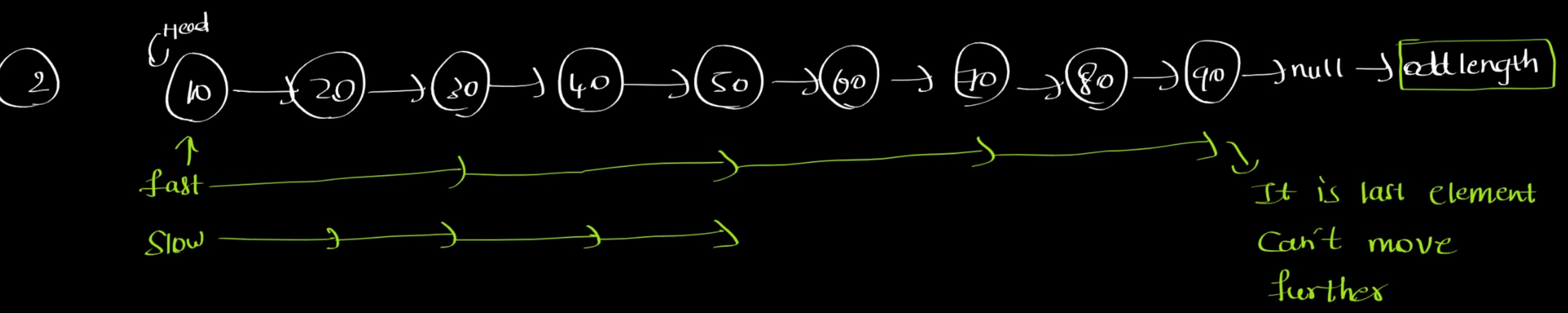
(2) index out of range → do nothing

code:

```
function deleteAtIndex(index)
{
    if( index<0 || index > size-1) return;
    if( index ===0)
    {
        this.head = this.head.next ;
    }
    else
    {
        let curr = this.head

        for(let i=0 ; i < index-1 ; i++)
        {
            curr = curr.next
        }
        curr.next = curr.next.next;
    }
    this.size--;
}
```

Middle Of linked list





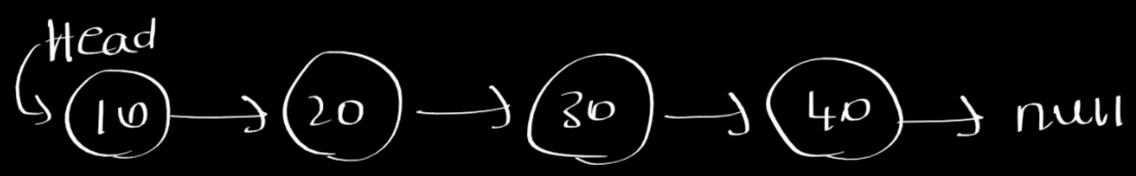
In even length, when we reach null
In Odd length, when we last element } we need to stop at these situations.

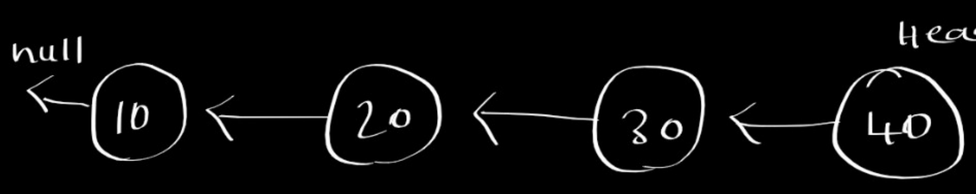
Algo: ① Maintain two pointers fast and slow
② until it reaches null or last element keep moving fast two steps and slow one step
③ if it is even length fast will reach null → if (fast !== null)
Otherwise fast will reach last element if (fast.next and !== null)
④ break loop return slow that will be middle element.

Code:

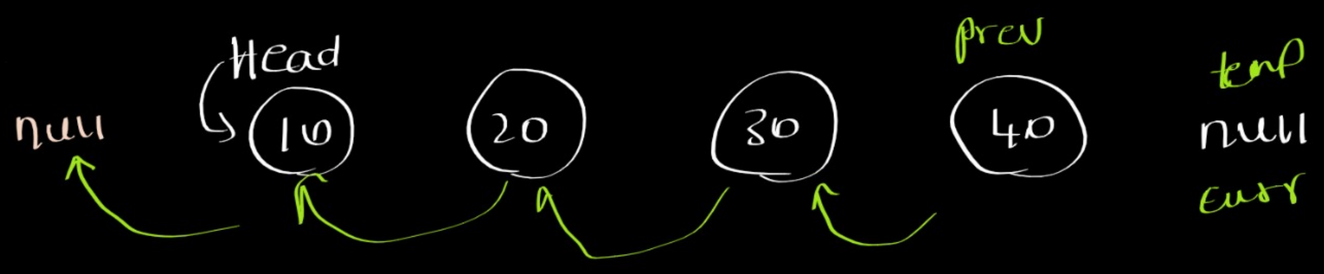
```
function middle(head)
{
    let slow = head;
    let fast = head;
    while (fast !== null && fast.next !== null)
    {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}
```

Reverse Linked List

Given: 

Output: 

we need to change the pointers.



Algo:
1. we need to maintain two ptrs Current and prev.
2. Current points to head and initially previous value is null
3. we need to change curr.next pointer to previous but if we do that we lose access to next pointer/node. so we need to maintain temporary variable.
4. we will move the temp to curr.next initially so that we don't lose access
5. now we without worry we can connect Current.next with previous
6. move the previous to current
7. now current to temp
8. now do 3 to 7 repeatedly until current becomes null.
9. finally prev will point the head at last. we can store it in head or directly return previous as it has the access.

Code:

```
function reverseLinkedList (head)
{
    let prev = null;
    let curr = this.head;
    while (curr)
    {
        let temp = curr.next;
        curr.next = prev;
        prev = curr;
        curr = temp;
    }
    head = temp
    return head;
}
```

Linked list cycle: Hash Table

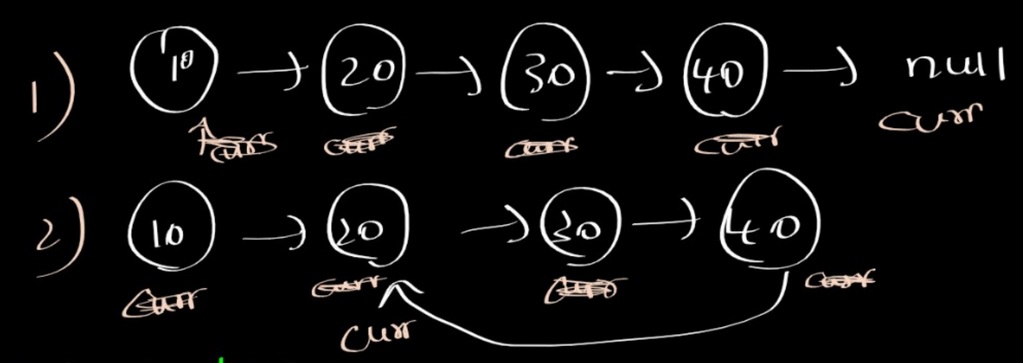
1) 10 → 20 → 30 → 40 → null (no cycle)

2) 10 → 20 → 30 → 40 (has cycle)

Algo:
1. maintain current pointer to traverse
2. Also maintain a hashtable/set to keep track of visited nodes.
3. first check whether the current element present in set if yes, then cycle exists we need to return true.
4. if not, add that element into set and move the current ptr to next node
5. Do 3 and 4 until it reaches null if it cannot reach it will return true if already element is present if it reach null while loop condn will become false then we can return false.

Code

```
function LinkedListCycle(head)
    {
        let seenNodes = new Set();
        let curr = head;
        while (curr)
        {
            if (seenNodes.has(curr)) return true;
            SeenNodes.add(curr);
            curr = curr.next;
        }
        return false;
    }
```



1) seenNodes (10, 20, 30, 40) false returned

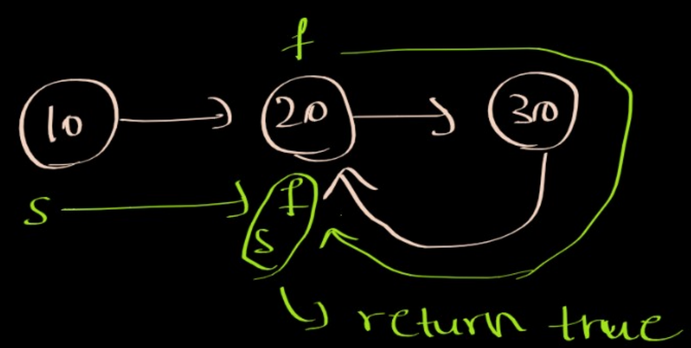
2) seenNodes (10, 20, 30, 40) true returned.

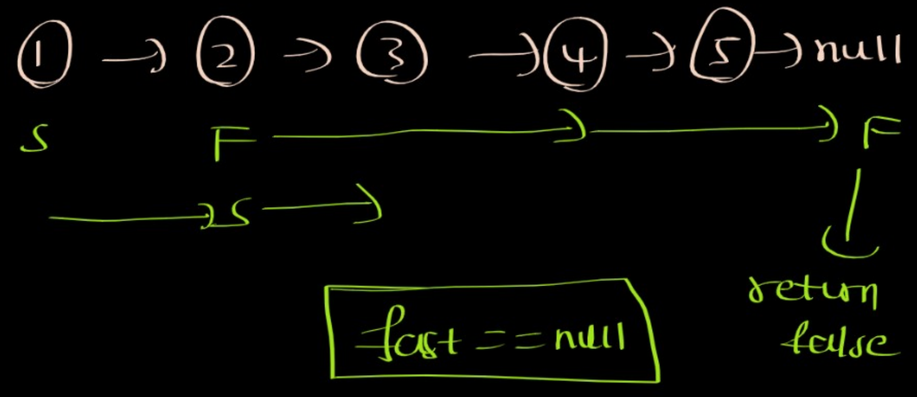
⇒ As we are traversing all nodes → time complexity → O(n)

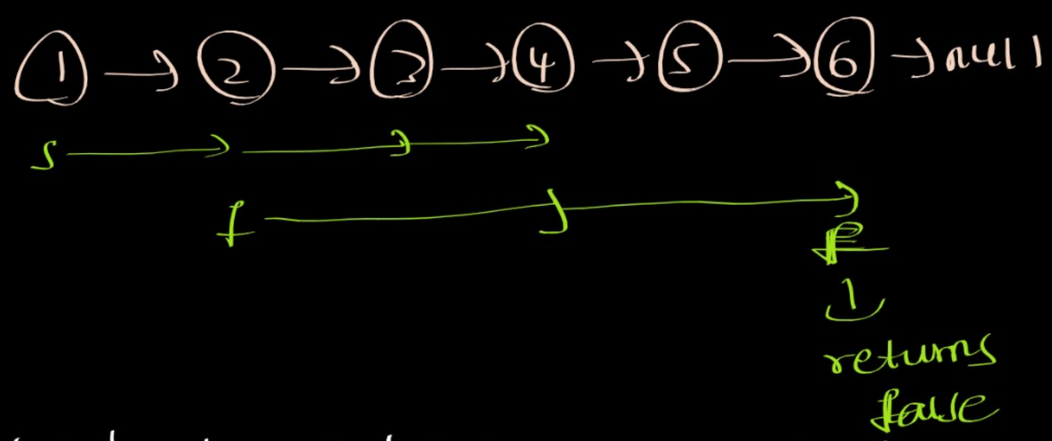
As we storing all nodes in set → space complexity → O(n)

Linked list Cycle - floyd Algorithm

Slow and fast pointer Approach: We maintain fast and slow pointer we move fast pointer faster than slow at some point both will reach (or) equal to each other then we can say linked list cycle exist.







Algo: (1) maintain two pointers slow and fast.

(2) slow at head, fast at head.next (as we don't want to make equal then on first instance)

(3) until both become equal keep on moving slow one step and fast two steps.

(4) if fast reaches null (or) fast.next is null return false. (No cycle)

(5) if both become equal break the loop, return true. (cycle exists)

Corner case: if there is an empty linked list return false.

Code:

```
function linkedListCycle(head) {
  if (head ===null) return false
  let slow = head
  let fast = head.next
  while (slow !== fast)
  {
    if (fast ===null || fast.next === null) return false;
    fast = fast.next.next;
    slow = slow.next;
  }
  return true;
}
```

Palindrome Linked List

(1) → (2) → (2) → (1) → null → palindrome

(1) → null → palindrome

(1) → (2) → (3) → (2) → (1) → null → palindrome

Approach 1: Brute force: (1) convert linked list into array
(2) Now check the array is palindrome or not.

Code:

```
function palindrome(head)
{
  let arr = [];
  let curr = head;
  while (curr)
  {
     arr.push(curr.val);
     curr = curr.next;
  }
  if (arr.length <1) return false;
  for(let i=0; i< Math.floor(arr.length/2); i++)
  {
       if (arr[i] === arr[arr.length-i-1])
       {  continue;
       }
       else {
          return false;
       }
  }
  return true
```

Converting LL to array

checking palindrome or not.

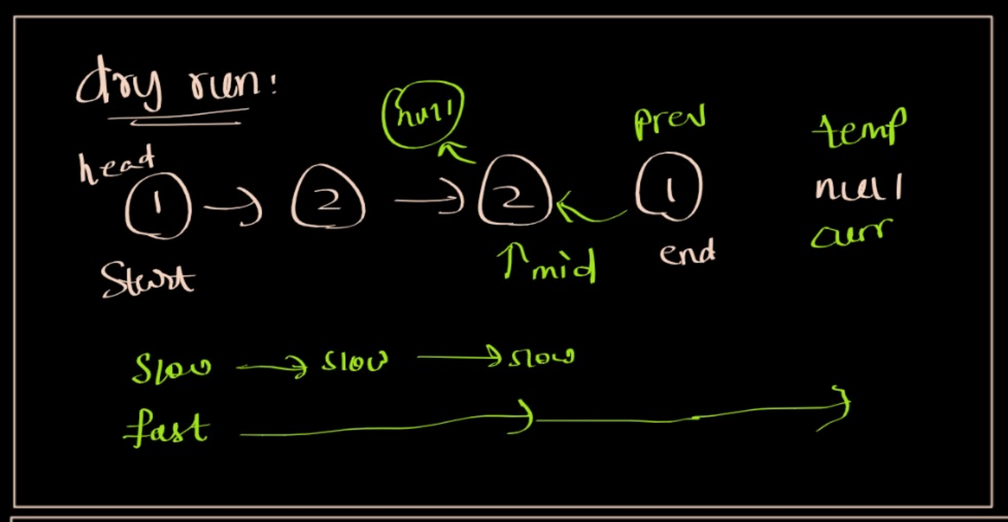
Time Complexity – O(n)
Space Complexity - O(n)

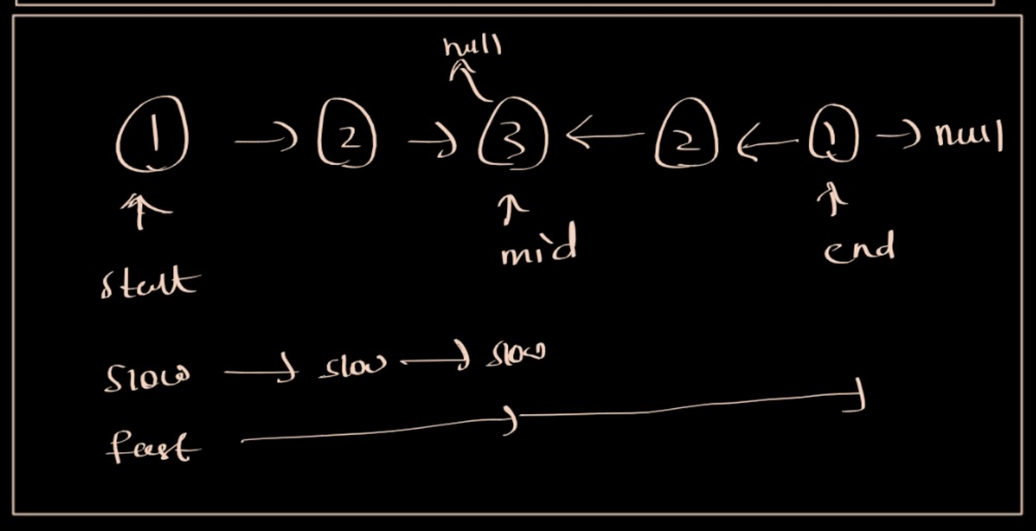
* **Approach 2:** Instead of using an array we can use another approach:

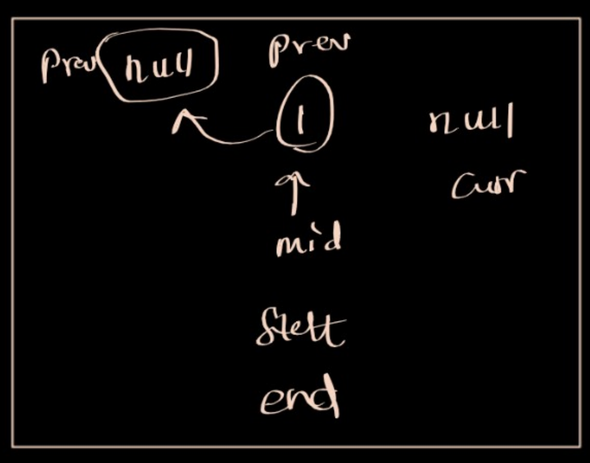
(1) find the middle node of linked list
(2) reverse the linked list from middle
(3) use start and end pointers
(4) Compare start and end values (not nodes) if not equal, not a palindrome
(5) if equal, continue the loop until reversedList becomes null
(6) if reversedList (end pointers) become null break the loop, return true (palindrome)

Code:

```
function palindrome(head)
{
  //finding the middle node
   let slow = head
   let fast = head
   while( fast && fast.next)
   {
       slow = slow.next;
       fast = fast.next.next;
    }
  // reversing the second half
    let prev = null;
    let curr = slow;
    while (curr)
    {
      let temp = curr.next;
       curr.next = prev;
        prev = curr;
        curr = temp;
    }
  // checking palindrome
   let start = head;
   let end = prev;
   While (end)
   {
      if (start.val !== end.val)
      { return false;
       }
       start = start.next;
        end = end.next;
    }
   return true;
```



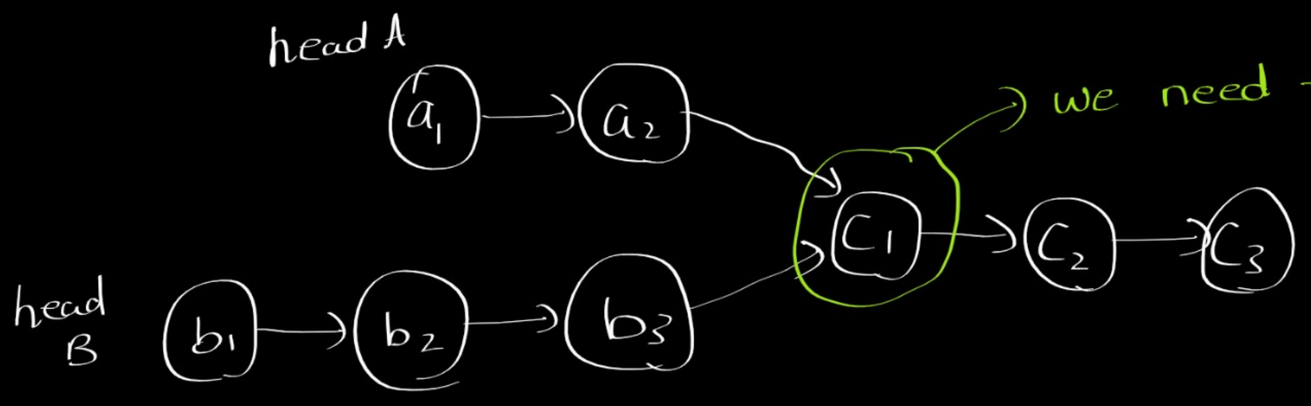




Start moves to undefined
end moves to null

returns true.

Intersection of two linked lists



Approach 1: Check each and every element of head A in head B one by one when you find both nodes are same return it.

Code:

```
function intersection(headA, headB)
{
  while(headA)
  {  let ptr=headB
     while( ptr )
     {
        if( ptr === headA)
          return headA;
          ptr = ptr.next;
     }
  } headA=headA.next;
  return null;
}
```

headA - m (size)
headB - n (size)

Time Complexity - $O(m \times n) \approx O(n^2)$
Space complexity - $O(1)$

⇒ we can optimize using another approach.

Approach2: using a hashmap to fetch (because it is O(1))

⇒ use a set and store all values of headB
⇒ Now check each and every node of headB one by one if it is present in set/hashmap
⇒ if present return node, otherwise return null after exhausting all nodes in headA.

Code:

```
function intersection (headA, headB)
{
   let store = new Set();
   while (headB)
   {
      store.add(headB);
      headB = headB.next;
   }

   while (headA)
   {
     if (store.has(headA)) return headA;
      headA = headA.next
   }
  return null;
}
```

headA - m
headB - n

Time complexity - $O(n+n) \approx O(n)$
Space complexity - $O(n)$

Remove Linked List Elements

Given : Head 1 → 2 → 3 → 6 → 4 → 5 → 6 → null Val = 6

Output : Head 1 → 2 → 3 → 4 → 5 → null

Algo: ① we will check the next node is equal to val

(if (prev.next.val === val) → prev.next = prev.next.next)

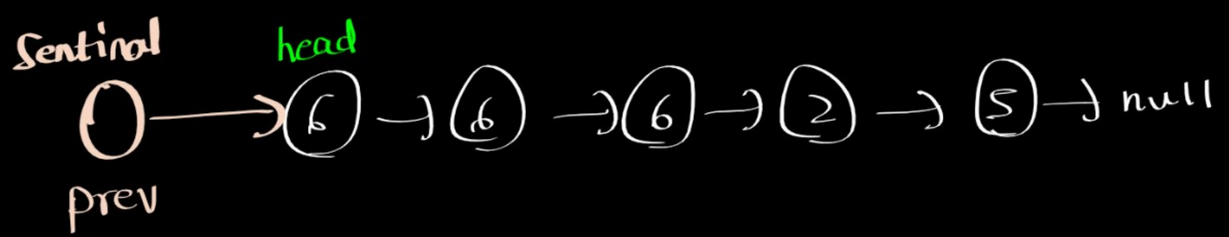
⇒ Always remember whenever we are deleting we need to maintain a prev element pointer.

⇒ But if we have :-

6 → 6 → 6 → 2 → 5 → null
prev

Here the first element itself is an element but can't delete it.

So, here we have something called **sentinal node** which actually works like a guard before prev



⇒ we will point sentinel node to front of linked list and it act as as previous element.

⇒ we will check prev.next is equal to val. if yes, point to next to next (prev.next = prev.next.next)

⇒ Otherwise move prev to next until prev (or) prev.next exists.

⇒ finally return Sentinel.next which is pointing to head.

Code:

```
function removeElements (head, val)
{
  let Sentinel = new ListNode();
  Sentinel.next = head;
  prev = Sentinel;
  while (prev && prev.next)
  {
    if (prev.next.val === val)
    {
      prev.next = prev.next.next;
```

```
function ListNode(val, next) {
 *    this.val = (val===undefined ? 0 : val)
 *    this.next = (next===undefined ? null :
next)
 * }
```

```
        }
        else
        {
            prev = prev.next;
        }
    }
    return sentinel.next;
```

Remove n^{th} node from end of list - Two pass

given : 1 → 2 → 3 → 4 → 5 → null (delete this) n=2

output : 1 → 2 → 3 → 5 → null

Algo:
1. find the length of list by traversing all elements (1 pass)
2. find the previous position so that we can delete the required node.
3. "start" prev "from sentenel node" and reach/traverse to prev position
4. now delete with `prev.next = prev.next.next`

Code:

```
function removeNthFromEnd(head, n)
{
    let sentenel = new ListNode(); } creating sentenel node
    sentenel.next = head;
    let prev = sentenel; → pointing prev to sentenel
    let length = 0;
    let curr = head;
    while (curr)                    } → calculating length of LL
    {
        curr = curr.next;
        length++;
    }
    let prevpos = length - n; → calculating prev pos to delete the required node
    for(let i=0; i < prevpos ; i++)
    {                               } → reach to prevpos
        prev = prev.next;
    }
    prev.next = prev.next.next; → delete the required node
    return sentenel.next; → return head.
```

time complexity - O(n+n) - O(n)
space complexity - O(1)

Remove n^{th} node from end of list - One pass, 2 pointers

⇒ Now we don't want to traverse through LL 2 times.

⇒ To do that let's maintain two pointers.

Algo: → main aim is to maintain gap of n between two pointers.

1. Create sentinel node
2. Start first pointer with gap of n from second pointer.
3. move first and second pointers until first reaches last node.
4. As we have gap of n, second pointer automatically reach the previous position
   (i mean, to delete required node we need to maintain prev before it = that position)
5. Now just delete the second.next
6. return head.

Code:

```
function removeNthFromEnd(head, n)
{
    let sentinel = new ListNode( );
    Sentinel.next = head;
```
→ creating sentinel node

```
    let first = sentinel;
    let second = sentinel;
    for(let i=0; i<n; i++)
    {
        first = first.next;
    }
```
→ maintaining gap of n b/w first and second

```
    while (first.next)
    {
        first = first.next;
        second = second.next;
    }
```
→ until first reaches last node move both pointers

```
    second.next = second.next.next;
```
→ delete the required node

```
    return sentinel.next;
```
→ return head.

Remove Duplicates from Sorted list

Ex: Head → (1) → (2) → (2) → (3) → (4) → (5) → (5) → null

Algo:
1. Start from head check whether next value and current value is equal or not.
2. if yes, move the next to (next to next) basically deletion.
3. if not, move the current pointer to next node.
4. Continue like this until current pointer becomes null (or) Current pointer next node is null.

Code:

```
function removeDuplicates(head)
{
    let curr = head;
    while (curr && curr.next)
    {
        if (curr.val === curr.next.val)
        {
            curr.next = curr.next.next;
        }
        else {
            curr = curr.next;
        }
    }
    return head;
}
```

T(c) - O(n)

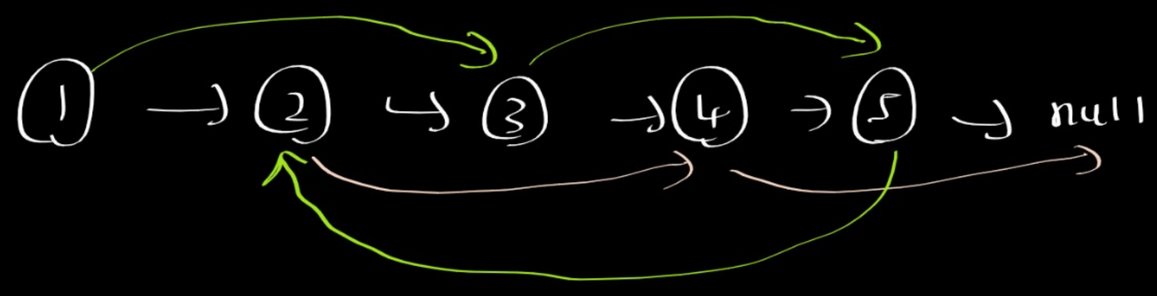
S(c) - O(1)

Odd Even Linked List

Input: Head → (1) odd → (2) even → (3) odd → (4) even → (5) odd → null

Output: (1) → (3) → (5) → (2) → (4) → null

Algo:
1. we need to remove the even elements inbetween initially then connect the last odd element to starting of even element.



⇒ maintain even, odd, evenstart pointers initially

=> move even and odd pointers one step ahead to point to the even and odd pointers according and also move the pointers

=> do this until even.next and odd.next exists

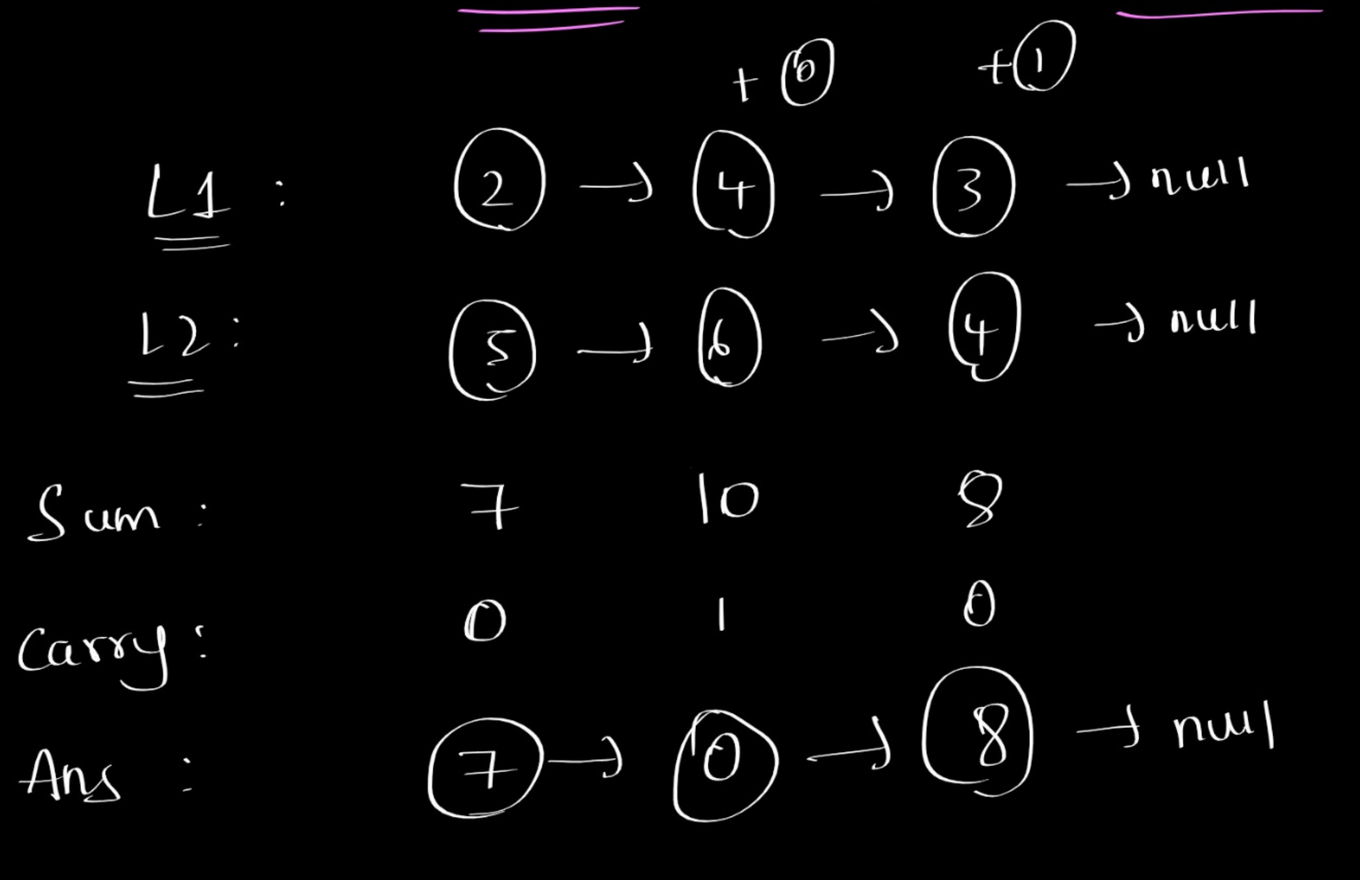
=> finally point the last odd element to evenstart.

Code:

```
function EvenOddList(head)
{
    if (head===null) return head;   // corner case
    let odd = head;
    let even = head.next;
    let evenstart = even;
    while (even.next && odd.next)
    {
        odd.next = odd.next.next;
        even.next = even.next.next;
        odd = odd.next;
        even = even.next;
    }
    odd.next = evenstart;
    return head;
}
```

T(c) - O(n)
S(c) - O(1)

Add Two Numbers



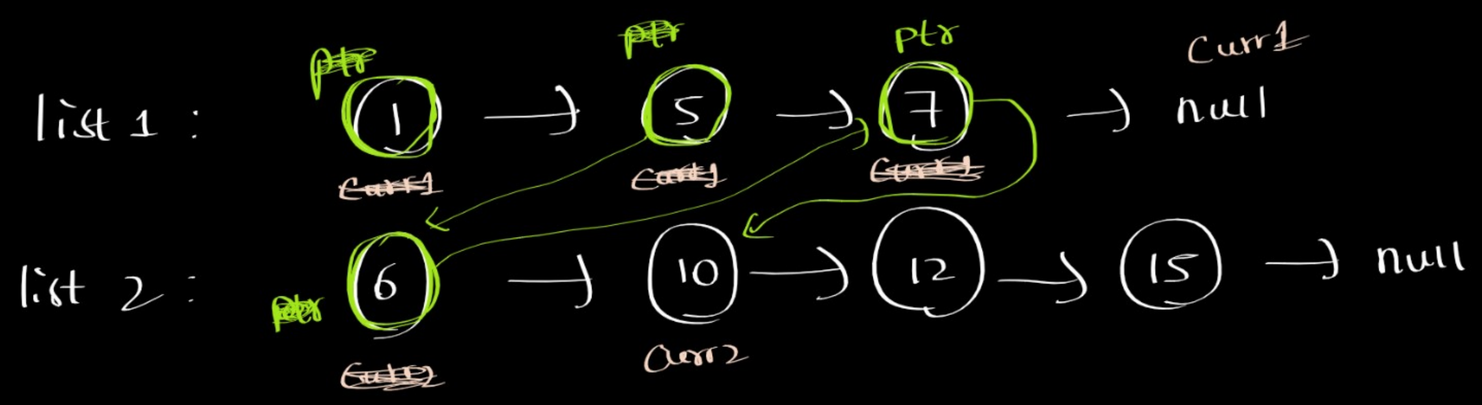
Algo: (1) find the sum if exits
(2) find the carry and the digit
(3) Add that digit to the newly created linked list.
(4) continue like this until l1, l2 and carry becomes 0.

Code:

```
function addNums(l1, l2)
{
    let dummyNode = new ListNode();
    let ans = dummyNode;
    let carry = 0
    while (l1 || l2 || carry)
    {
        let sum = (!l1) ? 0 : l1.val + (!l2) ? 0 : l2.val + carry;
        carry = Math.floor(carry/10);
```

```
let digit = sum%10;
let newNode = new ListNode(digit);
dummyNode.next = newNode;
dummyNode = dummyNode.next;

    l1 = l1 && l1.next;
    l2 = l2 && l2.next;
}
return ans.next;
```

Merge two sorted lists



Algo:

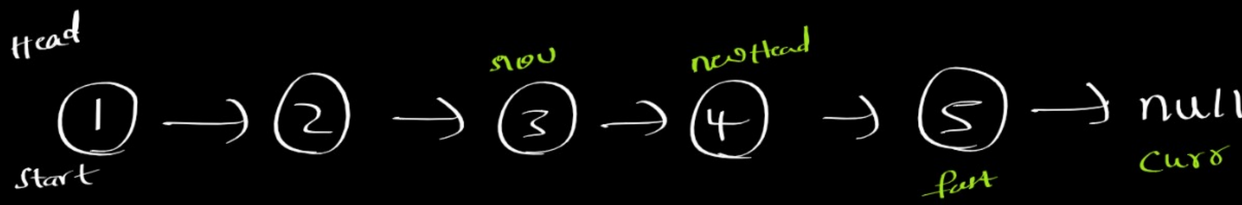
1. Create a dummy pointer
2. It will be point to next smaller value (1, 2, 3 .. etc)
3. If we find the smaller value move it to next value and also the pointer.
4. Continue like that until either of the list becomes null.
5. And according to which list is exhausted point the next of ptr to remaining nodes.

Code:

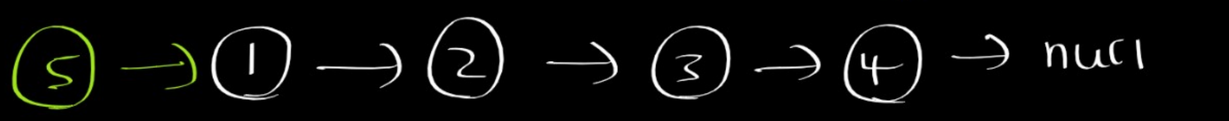
```
function mergeSortedLists(list1, list2)
{
    let curr1 = list1;
    let curr2 = list2;
    let ptr = new ListNode();
    let head = ptr;
    while(curr1 && curr2)
    {
        if (curr1.val < curr2.val)
        {
            ptr.next = curr1;
            ptr = ptr.next;
            curr1 = curr1.next;
        }
```

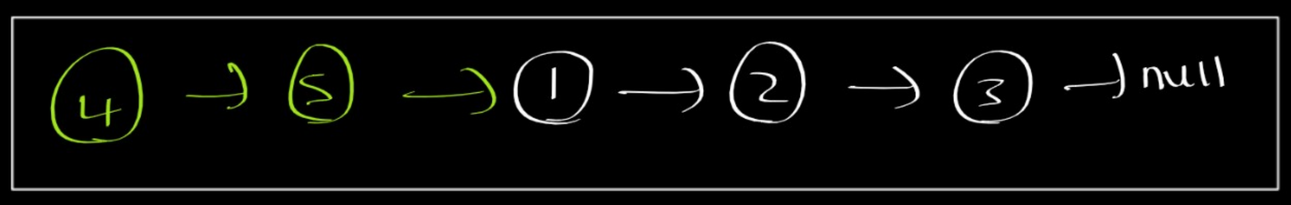
```
else
{
        ptr.next = curr2;
         ptr = ptr.next;
        curr2 = curr2.next;
}   }

(!curr1) ? ptr.next = curr2 : ptr.next = curr1;

return head.next;

}
```

Rotate list

given: 

K = 7 (or) 2

O/P: 



Algo: (1) reach K steps from last (or) end of linked list.

(we need to use slow and fast approach refer delete n^{th} node from last problem)

(2) when we reach make it as a new head

(3) And attach the last node to old head node.

Code:

```
function rotateList(head, K)
{  if(!head || !head.next) return head; → corner cases
   let fast = head;
   let slow = head;
   let curr = head
   let length = 0;
   while(curr)
   {
      length++;                    } → Calculating length
      curr = curr.next;
   }
   for(let i=0 ; i<(K%.length) ; i++)
   {                               } → move fast ahead of slow with
       fast = fast.next;             gap of (K%.length)
```

```
}
while (fast.next)
{
    slow = slow.next;
    fast = fast.next;
}
```
→ move fast to last element so that slow reaches to required element

```
fast.next = head;
```
→ attach the last element to Old head.

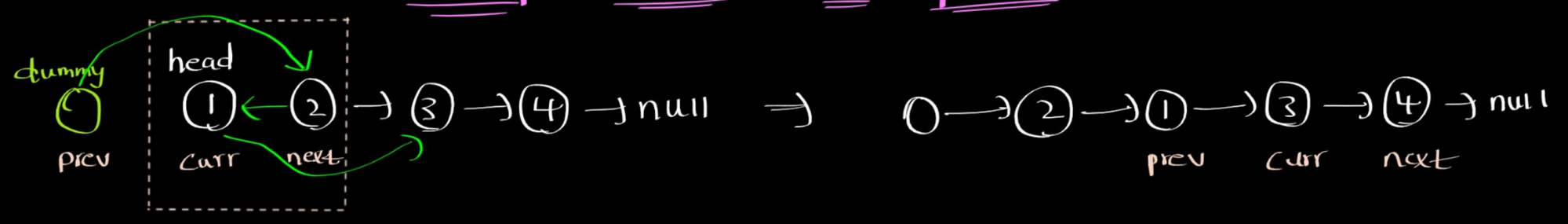
```
let newHead = slow.next;
```
→ mark new Head

```
slow.next = null;
```
→ make slow.next to null

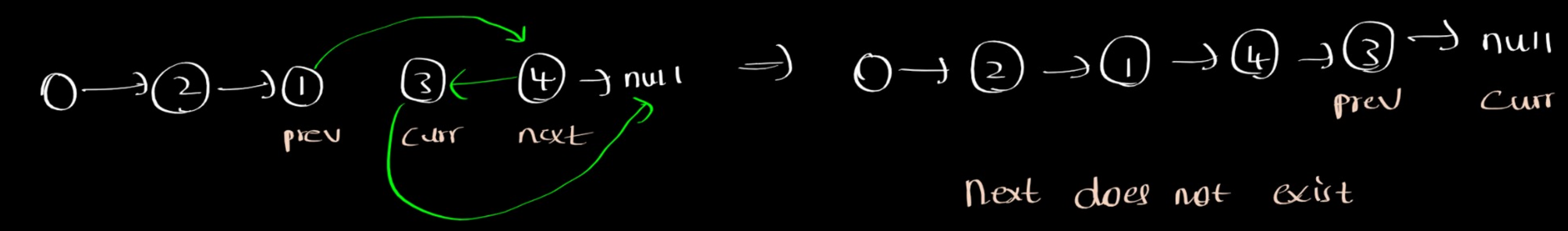
```
return newHead;
```
→ return new Head.

Swap Nodes in pairs

1st Itr:



2nd Itr:



Next does not exist
So, we will stop here and return dummy.next.

Code:

```
function SwapNodes(head)
{
    if(!head || !head.next) return head;
```
→ handling Corner cases

```
    let dummy = new ListNode();
    let prev = dummy
    let curr = head;
    let nxt = head.next;
```
} marking prev, curr and nxt

```
    while(curr && next)
```
→ until curr and next exists

```
    {
        prev.next = nxt;
        curr.next = nxt.next;
        nxt.next = curr;
```
} Swapping the nodes

```
        prev = curr;
        curr = prev.next
        nxt = curr && curr.next;
```
} moving pointers for next iterations

```
    }
    return dummy.next;
```
→ returning the starting point/head.

Recursive Approach

Code:

```
function swap(head)
{  if(!head || !head.next) return head;
   let left = head;
   let right = head.next;
   left.next = swap(right.next);
   right.next = left;
   return right;
}
```

Recursive Tree